

Moderne Datenbanken: Setup Guide

**The course stack: MongoDB, CockroachDB, Elasticsearch
and PHP in Docker, WS 2026/27**

Moderne Datenbanken

September 28, 2026

Contents

Contents	2
1 The Course Stack	3
1.1 Setting Up the Course Stack	3
1.1.1 Step 1: Docker	3
1.1.2 Step 2: The Stack	4
1.1.3 Step 3: First Start	4
1.1.4 Step 4: The PHP Libraries	4
1.1.5 Step 5: The Check	5
1.1.6 Step 6: MongoDB Compass	5
1.1.7 Stopping and Resetting	5
1.1.8 Troubleshooting	5
1.1.9 Final Check	6
1.1.10 Additional Resources	6
2 Getting Help	7

1 The Course Stack

This guide installs everything you need for *Moderne Datenbanken*: Docker, the course stack with its three database systems and PHP, and MongoDB Compass. Work through it before the first session; it ends with a check that the first session repeats.

1.1 Setting Up the Course Stack

You need this before the first session

- ▶ **Docker:** Docker Desktop on Windows or macOS, Docker Engine with the compose plugin on Linux, with **at least 6 GB of memory** for containers
- ▶ **The course stack** `mdb3-stack.zip` from Moodle: MongoDB, CockroachDB, Elasticsearch, Kibana and PHP in one docker compose file, plus the dataset
- ▶ **MongoDB Compass**, the free desktop client
- ▶ About **3 GB of disk** for the images and **20 minutes**, most of it download time

Everything in this course runs on your own machine. There is no cloud account, no shared server and no installation of a database on your operating system: the stack starts the three database systems and a PHP container, and stops them again when you are done. Every session and every homework sheet assumes the stack is running. The first session starts with a fifteen-minute check that it does; do the steps below **before** the session, so the check is a check.

Order of work

1. Install Docker, give it memory
2. Unpack the stack
3. First start
4. Install the PHP libraries
5. Run the check
6. Install Compass and connect

1.1.1 Step 1: Docker

Windows

1. Install **Docker Desktop** from <https://www.docker.com/products/docker-desktop/>. It uses WSL 2; the installer switches it on and asks for a restart.
2. Start Docker Desktop once and wait until the whale in the tray stops animating.
3. Open **Settings** → **Resources**. With the WSL 2 backend, memory is managed by Windows; if the stack later reports that Elasticsearch or CockroachDB were killed, create a file `%UserProfile%\wslconfig` with the two lines `[wsl2]` and `memory=8GB`, then `wsl --shutdown` in a terminal and start Docker Desktop again.

macOS

1. Install **Docker Desktop** for your chip (Apple silicon or Intel) from the same page. All images of the stack exist for both.

2. Open **Settings** → **Resources** (the gear icon top right, then **Resources** in the left list) and set the **Memory** slider to at least 6 GB, 8 GB if you have 16 GB or more. **Apply & restart**.

Linux

1. Install **Docker Engine** and the **compose plugin** from <https://docs.docker.com/engine/install/>, then add your user to the docker group and log in again. Docker Desktop for Linux works too.
2. No memory setting is needed; containers use what the machine has.
3. Kernel 6.19 or newer: the stack pins MongoDB 8.2 on purpose, because 8.0 and 8.3 refuse to start on these kernels. Do not change the image tag.

1.1.2 Step 2: The Stack

1. Download `mdb3-stack.zip` from Moodle and unpack it somewhere with a short path and no spaces, for example `C:\mdb3` or `~/mdb3`.
2. You get one folder `docker` with `docker-compose.yml`, a `php` folder (the PHP image), `scripts`, and `app`, which is the folder the PHP container serves. `app/data/AnimalCrossing` is the dataset; `app/stack.php` is the stack's own check, the one program you run before the course starts.

Keep every file of the course inside `app`: it is the only folder the PHP container can see, and `docker compose exec php php file.php` runs a file that lies there.

1.1.3 Step 3: First Start

In a terminal, inside the `docker` folder:

```
docker compose up -d --build
```

The first run downloads the images (MongoDB, CockroachDB, Elasticsearch, Kibana, about 2.5 GB) and builds the PHP image (a few minutes). Later starts take seconds. `docker compose ps` shows five containers; `mongodb`, `cockroach1` and `elasticsearch` must say `healthy` after a minute, `mongo-init` exits on purpose after it has initialised the replica set.

1.1.4 Step 4: The PHP Libraries

The PHP container has the MongoDB and PostgreSQL extensions built in; the client libraries come through Composer, once:

```
docker compose exec php composer install
```

This creates `app/vendor`. You never edit anything in it, and you run it again only if `composer.json` changes.

Check the installation: `docker compose version` in a terminal must print a version. On Windows use PowerShell or the Windows Terminal, not the old `cmd`.

Listing 1.1: The stack folder

```
docker/
  docker-compose.yml
  README.md
  php/Dockerfile
  scripts/
  app/
    composer.json
    stack.php
    import_catalogue.php
    data/AnimalCrossing/*.csv
```

Listing 1.2: `docker compose ps`, after a minute

NAME	STATUS
cockroach1	Up (healthy)
elasticsearch	Up (healthy)
kibana	Up
mdb3-php	Up
mongodb	Up (healthy)

Port 27017 already in use? Some machines run a MongoDB of their own. Start the stack with `MONGO_PORT=27018` `docker compose up -d` (PowerShell: `$env:MONGO_PORT=27018` first) and use 27018 in Compass. Inside the stack nothing changes.

1.1.5 Step 5: The Check

```
docker compose exec php php stack.php
```

Three lines, one per database system. That is the whole check, and it is what the first session starts with. If a line is missing, the troubleshooting section below has the usual causes. `stack.php` opens the three connections and prints one line per system; in the sessions you write your own `connect.php`, one connection at a time, and every program of the course starts from that.

1.1.6 Step 6: MongoDB Compass

1. Install Compass from <https://www.mongodb.com/products/tools/compass> (free, all platforms).
2. Connect with the URI `mongodb://localhost:27017?directConnection=true` (or your `MONGO_PORT`). `directConnection` is needed because the stack's MongoDB is a one-member replica set that knows itself as `mongodb:27017`, a name only the containers can resolve.
3. You see the databases `admin`, `config` and `local`. That is a running, empty MongoDB. The course data arrives in the first session.

The two other systems have web consoles you will use later: the CockroachDB DB Console at <http://localhost:8085> and Kibana at <http://localhost:5601> (Kibana needs a minute to start). Nothing to configure now.

1.1.7 Stopping and Resetting

- ▶ `docker compose stop` pauses the stack and keeps all data; `docker compose start` resumes it.
- ▶ `docker compose down` removes the containers and keeps the data volumes.
- ▶ `docker compose --profile cluster down -v` removes everything, data included. You can always start from zero; the first session shows how the data comes back in one command.
- ▶ If the stack is too much for your machine, stop Kibana until *Search I*: `docker compose stop kibana` saves about 1 GB.

1.1.8 Troubleshooting

- ▶ **docker: command not found** or **Cannot connect to the Docker daemon**: Docker Desktop is not running, or (Linux) your user is not in the docker group.
- ▶ **Bind for 0.0.0.0:27017 failed: port is already allocated**: another MongoDB is running on your machine; use `MONGO_PORT` as above. The same message for 9200, 5601, 8084 or 8085 means another program uses that port; stop it or edit the left-hand port in `docker-compose.yml`.
- ▶ **elasticsearch or cockroach1 keeps restarting**: usually memory. Give Docker more (Step 1) or stop Kibana.

Listing 1.3: Expected output

```
MongoDB:      ok (ping=1)
CockroachDB:  CockroachDB
               CCL v26.2.6 (...)
Elasticsearch: 9.1.4
```

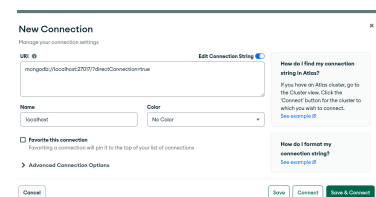


Figure 1.1: Compass, New Connection: the URI with `directConnection=true`, then Save & Connect.

- ▶ **MongoDB logs Linux kernel versions 6.19 and newer has a known incompatibility:** you changed the image tag; put `mongo:8.2` back.
- ▶ **Class "MongoDB\Client" not found:** Step 4 was skipped, or Composer ran on your machine instead of in the container. Run `docker compose exec php composer install`.
- ▶ **stack.php hangs on the CockroachDB line:** the container is still starting; `docker compose ps` must show it healthy. Wait a minute and retry.
- ▶ **Compass shows getaddrinfo ENOTFOUND mongodb:** the URI is missing `?directConnection=true`.

1.1.9 Final Check

Ready for the first session

- ☐ `docker compose version` prints a version
- ☐ `docker compose ps` shows `mongodb`, `cockroach1` and `elasticsearch` as healthy
- ☐ `docker compose exec php php stack.php` prints three lines
- ☐ Compass is connected and shows `admin`, `config`, `local`
- ☐ <http://localhost:5601> shows Kibana (optional today)

1.1.10 Additional Resources

- ▶ Docker Desktop settings and WSL 2 memory: <https://docs.docker.com/desktop/settings-and-maintenance/settings/>
- ▶ Compass connection strings: <https://www.mongodb.com/docs/compass/connect/>
- ▶ The stack's `README.md` in the `docker` folder repeats the commands of this guide in short form.

2 Getting Help

If you run into a problem the troubleshooting section does not cover, post it in the Moodle forum with the exact command you ran and the exact output, so others with the same problem find the answer. Bring it to the first session in any case; the first fifteen minutes are for exactly this.